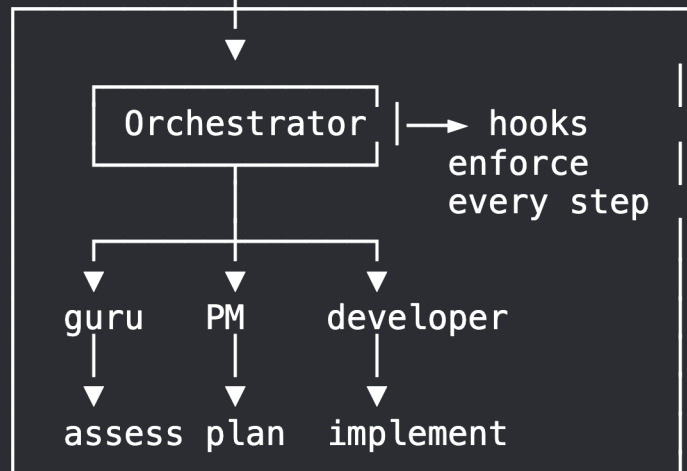


Arkadian

Multi-Agent AI Orchestration for Ark

User → arkadian "Fix VTX0 expiry in fulmine"



Problem 1

The Multi-Repo Reality

20 Repositories, 5 Languages, 1 Protocol

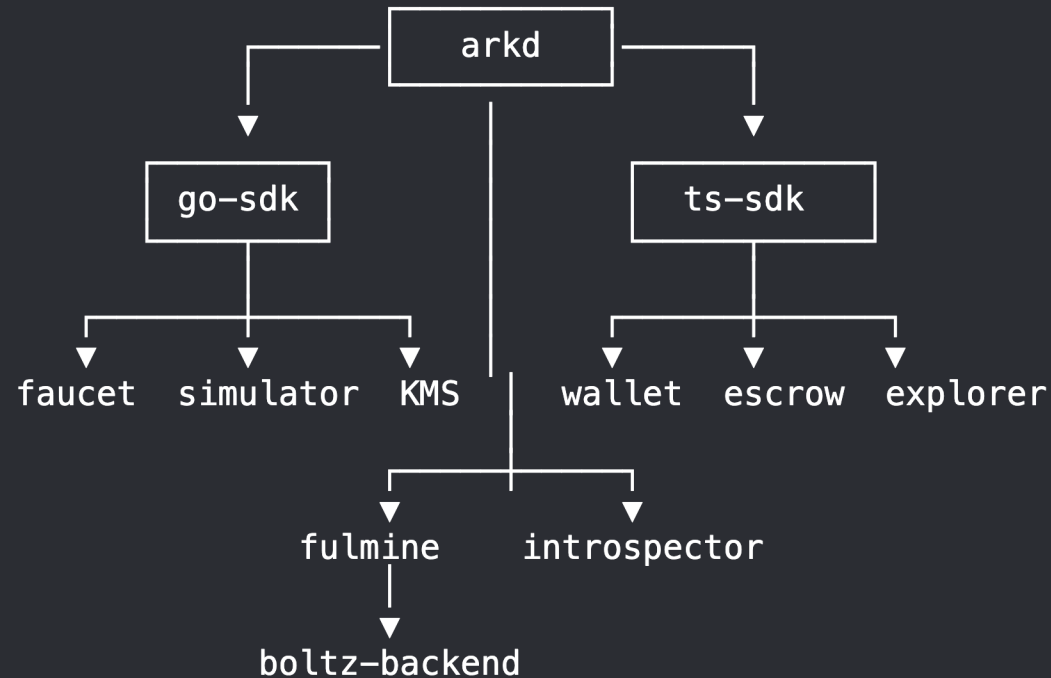
Our ecosystem isn't one repo. It's **20 interconnected projects**:

arkd (core server)	Go
— go-sdk, ts-sdk, rust-sdk, dotnet-sdk (4 client SDKs)	Go, TS, Rust, C#
— wallet (PWA)	TypeScript/React
— fulmine (Lightning wallet)	Go
— ark-faucet, ark-simulator	Go
— arkade-escrow	TypeScript/NestJS
— arkade-explorer	TypeScript/React
— arkade-assets (asset protocol)	TypeScript
— compiler + introspector	Rust + Go
— boltz-backend, boltz-swap	TypeScript/Rust
— ark-telemetry, ark-infra	YAML/HCL
— ark-docs	MDX

A single feature can touch **3-5 repos**. Lightning swap integration touches

`fulmine` + `boltz-backend` + `boltz-swap` + `wallet` + `ts-sdk`.

Cross-Repo Is the Norm, Not the Exception



When you ask an LLM to work on this, it needs to understand **where things are, how they connect, and which patterns each repo uses.**

The Documentation Registry – A Map of Everything

We built `docs/INDEX.md` – a machine-readable map of the entire ecosystem:

```
### fulmine
ID: fulmine
Type: Service/Bitcoin Wallet
Language: Go
Tags: wallet, lightning, swap, submarine-swap, chain-swap,
      boltz, bitcoin, vhtlc, delegator, opentelemetry
Triggers:
  ask_question: lightning swap, submarine swap, vhtlc, delegator
  develop: add swap feature, web ui, chain swap, delegator
  debug: swap failed, htlc issues, boltz errors
Dependencies: boltz-backend, Bitcoin node
```

Each project has a per-project `INDEX.md` with:

- Architecture docs, folder structure, testing guides
- Default doc sections to load per intent type
- Available scripts (`make test`, `docker compose up`, etc.)

Problem 2

The Context Engineering Challenge

What Is Context Engineering?

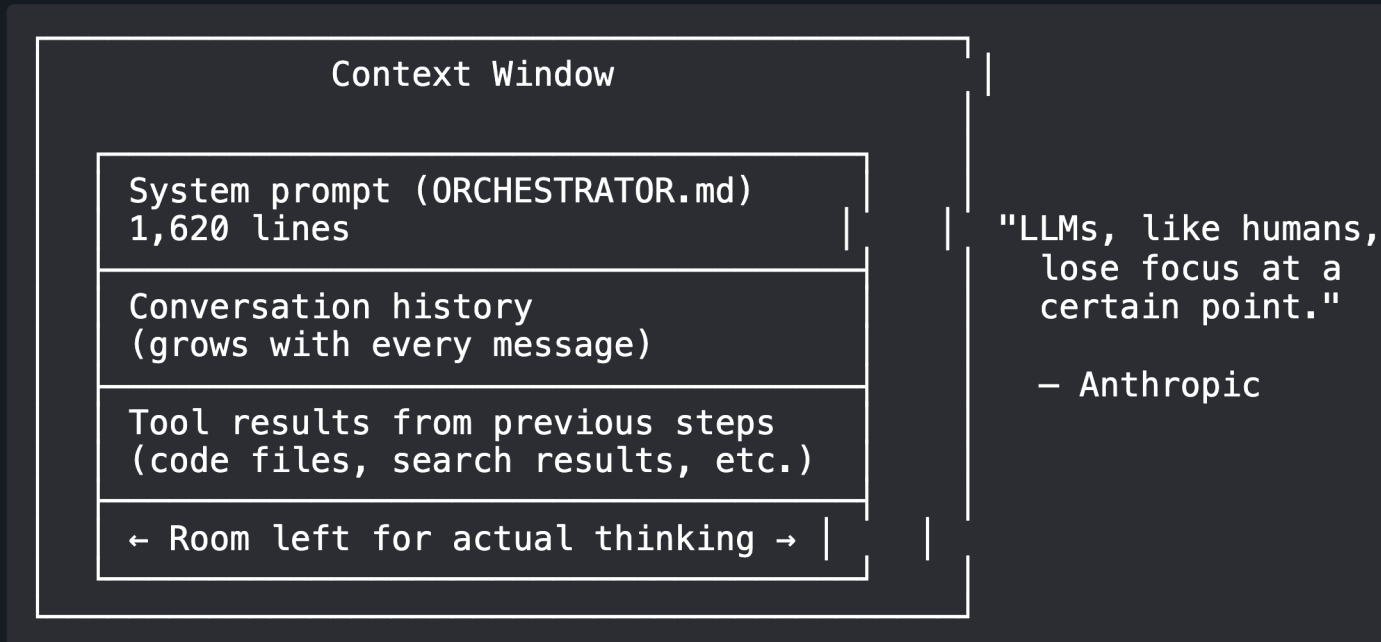
"The art of curating all tokens available to an LLM – system instructions, tools, external data, message history – to maximize the likelihood of a desired outcome."

– Anthropic, "Effective Context Engineering for AI Agents"

It's not just prompt engineering. It's about **what information enters the model's limited attention budget at each step.**

The core insight: **every token depletes a finite resource.**
More context doesn't mean better results – it means more noise.

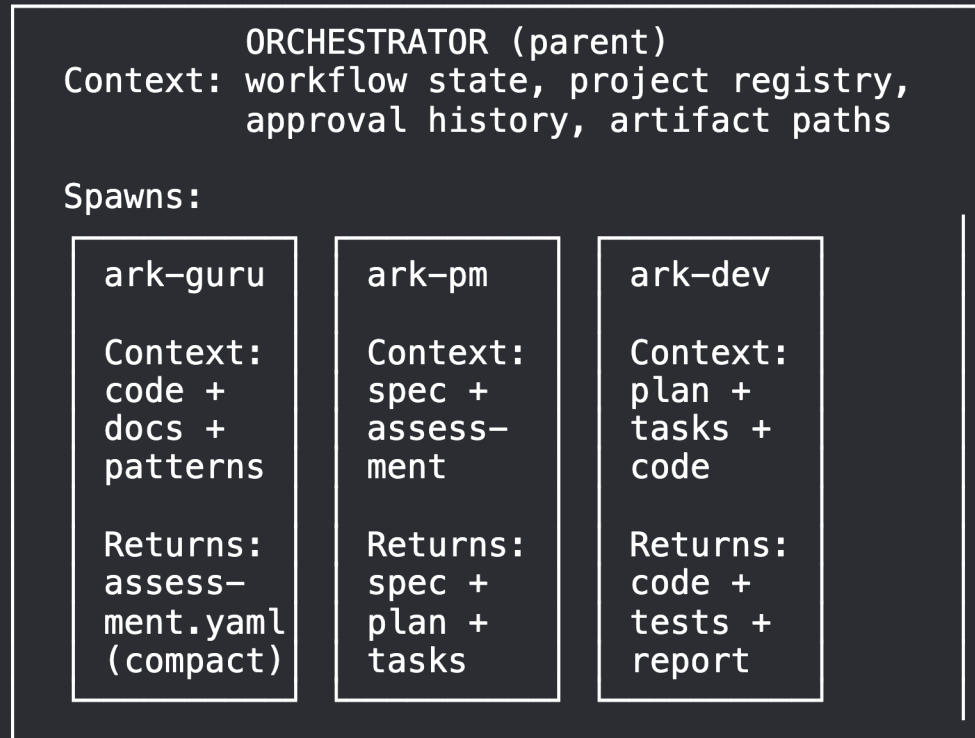
The Problem: Context Windows Fill Up



A single exploration of `arkd` can read 10+ files.
By the time you get to implementation, the model has **forgotten the plan.**

Our Solution: Sub-Agents With Isolated Context

Each agent gets a **clean context window** focused on its task:



Sub-agents explore **extensively** but return **condensed summaries**.

Tiered Context Loading

Not everything gets loaded at once. Context flows in 4 tiers:

Tier	Who Loads	What	When
1	Orchestrator	Master INDEX.md	Always, first action
2	Orchestrator	Per-project INDEX.md	After project selection
3	Agents	Doc sections (architecture, testing)	Via execution spec
4	Agents	Actual code files	As needed during task

The orchestrator **never reads code**. Only agents do.
This keeps the orchestrator's context clean for workflow decisions.

"The smallest set of high-signal tokens that maximize the likelihood of some desired outcome."

Problem 3

AI Doesn't Follow Workflows

The Discipline Problem

Ask an LLM to follow a 4-step process. What actually happens:

```
Intended:  explore → plan → implement → test → review
Reality:   "Let me just quickly fix this..."
           *skips exploration*
           *skips planning*
           *writes code on main branch*
           *forgets to test*
           "Done!"
```

LLMs are **eager to help** and **terrible at self-discipline**.
They optimize for appearing helpful, not for following process.
Telling the model "always follow the pipeline" in the prompt **is not enough**.
It will comply 80% of the time. The other 20% is where bugs ship.

Solution: OS-Level Hook Enforcement

We don't ask. We **enforce**. Hooks intercept every tool call:

Claude wants to call Task(ark-developer, ...)

PreToolUse hooks
(11 files, ~4,730
lines of code)

← TypeScript, runs via Bun
exit 0 = allow
exit 2 = BLOCK

Does assessment.yaml exist?
Does specs/ have plan.md?
Is workflow.yaml created?

YES

NO

agent runs

✗ BLOCKED

"Missing: artifacts/explore/
assessment.yaml"

The LLM literally cannot skip a phase. It's enforced at the OS level,
not in the prompt. The hooks run *outside* the model's control.

Real Error When Skipping Steps

✗ PIPELINE PREREQUISITE FAILURE

- ark-developer requires guru exploration first.
Missing: artifacts/explore/assessment.yaml.
Invoke ark-guru with context_intent: dev before ark-developer.
- Planning phase required but no PM specs found for fulmine.
Mandatory pipeline: ark-guru → ark-project-manager → ark-developer.

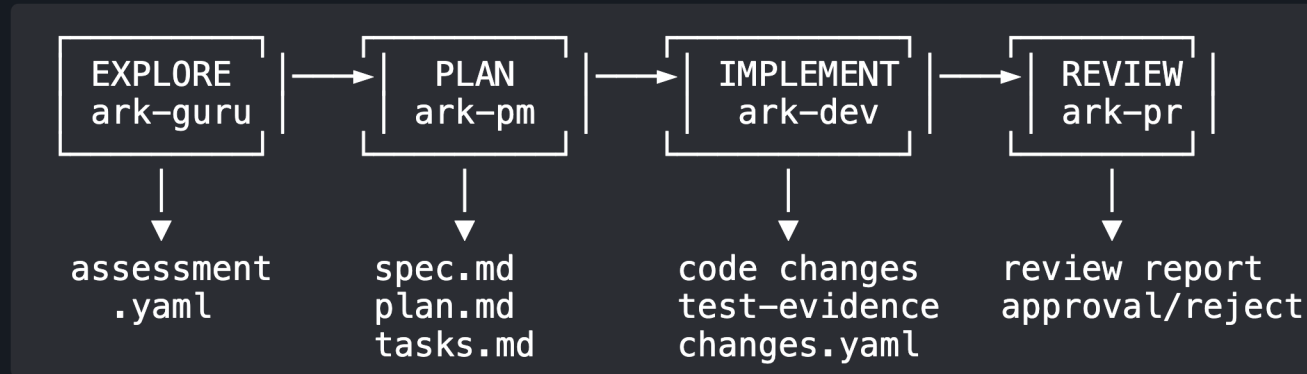
The mandatory pipeline is:
ark-guru (explore) → ark-project-manager (plan)
→ ark-developer (implement).
Each phase must complete and produce its artifacts before the next can start.

From `pre-agent-validator.ts` – this is a **real error**, not a warning.

Solution

How Arkadian Works

The Four-Phase Pipeline



Every phase ALWAYS executes. No conditional skipping. v5.0.0.

Artifact chain – each phase gets **ALL** prior artifacts:

- S2 (plan) gets: S1 assessment
- S3 (implement) gets: S1 assessment + S2 spec/plan/tasks
- S4 (review) gets: S1 + S2 + S3 – the complete picture

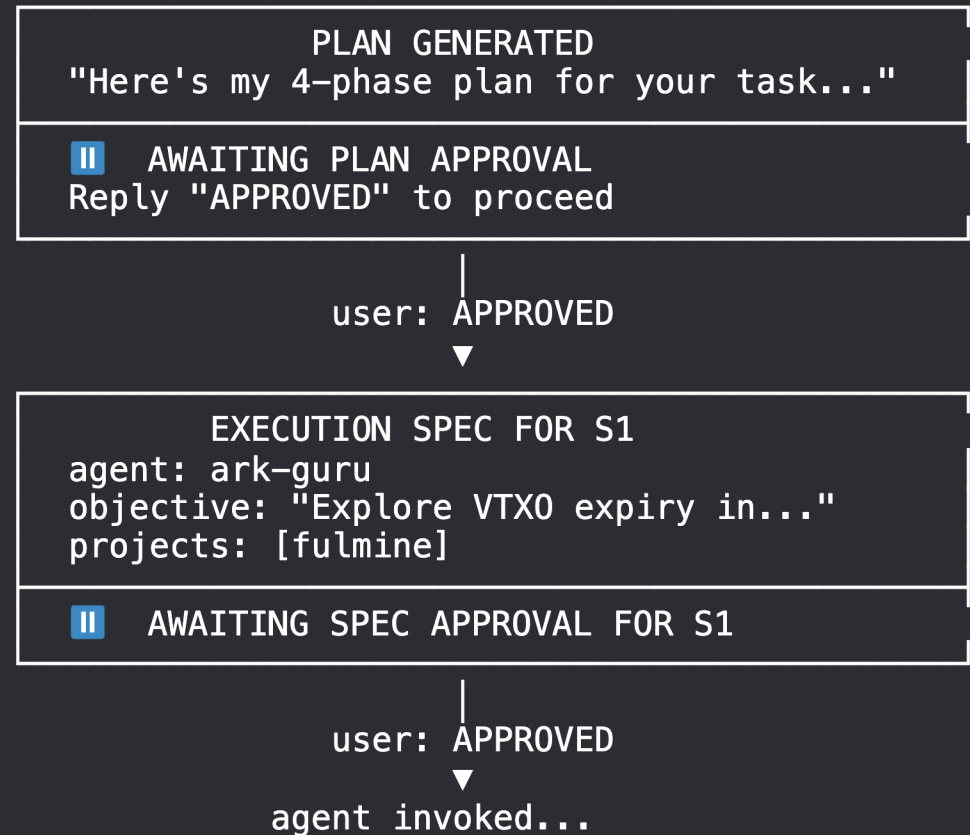
Artifacts: The Handoff Protocol

Agents are **stateless**. They don't remember previous phases.
Artifacts are how context flows between steps:

```
artifacts/explore/  
  assessment.yaml      ← complexity, affected files, risks  
  _result.json         ← machine-readable phase completion  
  
specs/fulmine/001-feature/  
  spec.md              ← requirements, user stories  
  plan.md              ← architecture decisions, approach  
  tasks.md             ← dependency-ordered breakdown  
  _result.json         ← phase completion proof  
  
artifacts/implement/  
  detailed_report.md   ← what was built and why  
  test-evidence.md     ← test commands + raw output  
  changes.yaml         ← branch, commits, files changed  
  _result.json         ← phase completion proof
```

Every `_result.json` is validated by post-agent hooks.
Missing artifacts = **hard gate failure** = retry or escalate.

3 Approval Gates (Human Stays in Control)



"APPROVED ALL" skips remaining gates (opt-in).

The Execution Specification

Every agent invocation starts with a machine-validated contract:

```
# --- BEGIN AGENT INPUT ---
step_id: "S3"
agent: "ark-developer"
objective: "Implement GetSwapEvents streaming RPC for fulmine"
user_request: "Add event streaming to fulmine wallet updates"
context_intent: "dev"
parent_session_id: "a1b2c3d4-..."

projects:
  - id: "fulmine"
    repo_source:
      repo_root: "${FULMINE_REPO}"
    doc_source:
      sections: ["system/architecture.md", "testing/how_to_test.md"]

artifacts_in:
  - path: "artifacts/explore/assessment.yaml"
    from_step: "S1"
  - path: "specs/fulmine/001-getswapevents/plan.md"
    from_step: "S2"

worktree_config:
  enabled: true
# --- END AGENT INPUT ---
```

Security Boundaries

```
// From orchestrator-guardrail.ts – the orchestrator is locked down
const ALLOWED_TOOLS = [
  'Task',          // Delegate to agents (ONLY allowed action)
  'Read',          // Read docs (restricted to ARKADIAN_DIR)
  'Write',         // Write files (restricted to ARKADIAN_DIR)
  'TodoWrite',     // Track workflow state
  'AskUserQuestion' // Clarify requirements
];

const BLOCKED_TOOLS = [
  'Bash',          // Orchestrator CAN'T run commands
  'NotebookEdit', // Orchestrator CAN'T edit notebooks
  'MultiEdit'      // Orchestrator CAN'T multi-edit code
];
```

The orchestrator **cannot run Bash** or touch project repos.

Worktree isolation (ark-developer)

All code changes go to `.worktrees/<branch>` – **never the main branch**.
The subagent guardrail blocks writes to main repo paths.

8 Specialist Agents

Agent	Role	Key Tools
ark-guru	Explore, analyze, Q&A	Read, Glob, Grep, WebSearch
ark-project-manager	Spec, plan, task breakdown	Read, Write, Skill
ark-developer	Implement + test	Read, Write, Edit, Bash
ark-pr-reviewer	Code review	Read, Grep, Bash
ark-observer	Telemetry debugging	Read, Bash, WebFetch
ark-researcher	Bitcoin/L2 research	WebSearch, WebFetch, Task
ark-progress-tracker	Progress reports	Read, Bash, WebFetch
claude-search-agent	Web research worker	WebSearch, WebFetch

Each has a **fixed tool allowlist**. **ark-guru** cannot run **Bash**.
ark-developer cannot use **WebSearch**. Enforced by hooks.

33 Domain Skills

Pre-loaded knowledge agents invoke on demand:

Domain	Skills
Protocol	ark-bitcoin-primitives, ark-musig2-signing, ark-vtxo-model
SDKs	ark-sdk-client-init, ark-sdk-payments, ark-sdk-settlement, ark-sdk-batch-session
Arkd	arkd-round-lifecycle, arkd-tree-construction, arkd-offchain-tx, arkd-grpc-api
Fulmine	fulmine-dev-loop, fulmine-vhtlc, fulmine-submarine-swap, fulmine-reverse-swap, fulmine-chain-swap, fulmine-batch-settlement
PM	pm-spec, pm-plan, pm-tasks, pm-analyze, pm-clarify, pm-checklist, pm-constitution
DevOps	arkd-dev-loop, ark-ops, ark-testing-patterns, ark-repository-patterns
Research	bitcoin-l2-research, browser-testing, ark-progress-tracking, ark-wallet-dev

Skills aren't static docs – they're **structured decision frameworks**

that guide agents through multi-step procedures.

Features

Sessions, Roles, Commands

Session Storage & Traceability

Every task creates a named, traceable session:

```
sessions/fulmine/  
  2026-02-16-implement-getswapevents-streaming-grpc-rpc/  
    ├── metadata.json          # who, when, what  
    ├── workflow.yaml          # approved plan (YAML)  
    ├── specs/  
    │   ├── S1.yaml            # exploration spec  
    │   ├── S2.yaml            # planning spec  
    │   ├── S3.yaml            # implementation spec  
    │   └── fulmine/001-feature/  
    │       ├── spec.md plan.md tasks.md  
    │       └── _result.json  
    └── artifacts/  
        ├── explore/ assessment.yaml _result.json  
        └── implement/ detailed_report.md test-evidence.md  
                      changes.yaml _result.json
```

Sessions are **named after completion** (UUID → friendly name).

Browse, diff, or resume any session at any time.

Session Resume

Pick up exactly where you left off:

```
# Resume by friendly name
arkadian --resume 2026-02-16-implement-getswapevents-streaming-grpc-rpc

# Resume by UUID
arkadian --resume a1b2c3d4-e5f6-7890-abcd-ef1234567890

# Resume in background (detached)
arkadian --resume 2026-02-16-implement-getswapevents -d
```

```
Session state analysis:
✓ explore:  COMPLETE (ark-guru)
✓ plan:    COMPLETE (ark-project-manager)
⏏ implement: IN PROGRESS (no _result.json)
Progress: 2/4 phases complete
Next phase: S3 (spec exists: S3.yaml)

Starting orchestrator in resume mode...
```

Hook copies old session files into fresh Claude session.
`_result.json` files determine which phases to skip.

Detached Mode (Fire and Forget)

Run the entire pipeline headlessly:

```
arkadian -d "Add GetRoundMetrics RPC endpoint to arkd"

Arkadian detached (PID 48291)
  Session: a1b2c3d4-...
  Dir:     sessions/a1b2c3d4-.../
  Output:  sessions/a1b2c3d4-.../output.log

  Monitor:
    arkadian status a1b2c3d4
    arkadian logs a1b2c3d4
```

- All 3 approval gates **auto-approved**
- Agent decisions auto-resolved (agent's preference wins)
- Full artifacts produced (identical to interactive)
- Hook enforcement still active
- Up to 3 retries on failure
- Combine with `--resume` to finish interrupted work overnight

Project Management Commands

Registry Management

```
# Analyze repo + add to registry
/add-project

# Update docs from new commits
/update-project

# Remove from registry + delete docs
/remove-project
```

SpecKit Workflow

```
/speckit.specify # Write spec
/speckit.plan # Generate plan
/speckit.tasks # Break into tasks
/speckit.implement # Execute tasks
/speckit.analyze # Cross-check
/speckit.clarify # Ask questions
/speckit.checklist # Quality gates
/speckit.constitution # Principles
```

Each command reads the repo,
generates
metadata (tags, triggers,
capabilities),
creates **INDEX.md** and doc
structure.

Session Analysis

Retry & Failure Recovery

When an agent fails, post-agent validation catches it:

```
=====
AGENT_VALIDATION: ark-developer (S3)
OUTCOME: failed
RETRY_ELIGIBLE: true (attempt 1 of 3)

HARD GATE FAILURES (2):
  [HG-IMPL-DEV-01] Missing detailed_report.md
  [HG-IMPL-DEV-03] Missing changes.yaml

RETRY GUIDANCE:
  Complete implementation and produce all 3 required
  artifacts before exiting.
=====
```

- Up to **3 retries** with structured feedback from the hook
- Retry context injected into next attempt (agent knows what failed)
- After 3 failures: **escalate to human** (never auto-retry forever)
- `_result.json` tracks attempt count for resume awareness

Assumption Challenging (Built Into Every Agent)

Agents don't blindly execute. They push back:

```
## ⚠ Assumption Challenge

**Assumption:** "Add a gRPC endpoint to delete all VTX0s"

**Why this may be incorrect:**
- VTX0s represent user funds – deletion violates Ark's security model
- VTX0s should only be removed through:
  1. User-initiated exit flows
  2. Natural expiry after timeout

**Evidence:** internal/core/domain/vtxo.go:85–120

**Recommended alternative:**
- Add a "mark expired" endpoint that transitions state without deleting the record
```

From the **actual ark-developer agent prompt** (not a demo).

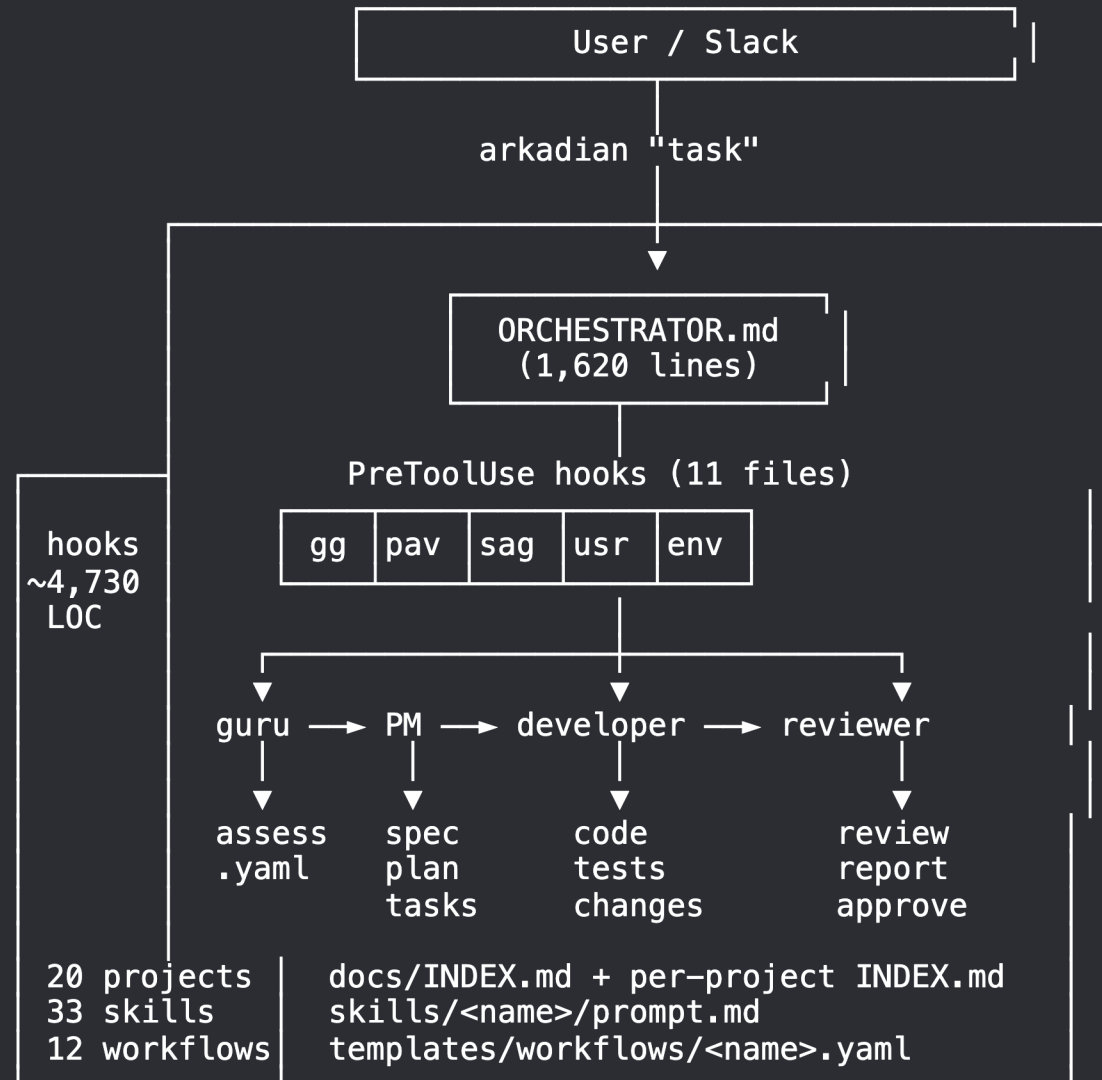
It's Live

Real Sessions From This Week

```
sessions/fulmine/  
  2026-02-13-add-event-stream-for-fulmine-wallet-updates/  
  2026-02-16-implement-getswapevents-streaming-grpc-rpc/  
  2026-02-17-add-vhtlc-script-column-and-subscription-handler/  
  2026-02-19-vhtlc-schema-migration-preimage-hash-to-id-pk/  
  2026-02-19-add-custom-preimage-support-to-getinvoice/  
  2026-02-19-add-preimage-return-to-payinvoice-method/  
  2026-02-19-investigate-grpc-reconnect-logic-after-server-rest/  
  
sessions/arkd/  
  2026-02-19-update-arkd-documentation-in-arkadian-registry/  
  2026-02-19-resume-bitcoin-mtp-expiry-implementation-workflow/  
  
sessions/arkadian/  
  2026-02-19-fix-detached-mode-and-resume-bugs/  
  2026-02-19-verify-resume-fixes-investigate-detached-yolo-mode/
```

Every session: full artifacts, specs, `_result.json`.
Every decision traceable. Every phase resumable.

The Full Picture



Try It

```
# Install
git clone <repo> && cd arkadian && make install

# Ask a question (routes to ark-guru)
arkadian "How does round finalization work in arkd?"

# Fix a bug (full 4-phase pipeline)
arkadian "Fix the VTX0 expiry handling in fulmine"

# Run overnight (detached mode)
arkadian -d "Add GetRoundMetrics RPC endpoint to arkd"

# Check status
arkadian status

# Resume where you left off
arkadian --resume 2026-02-19-fix-vtxo-expiry
```

Questions?

```
arkadian "your question here"
```

```
8 agents. 33 skills. 20 projects.  
Hook-enforced. Session-traceable.  
Resumable. Detachable.
```

```
One command to install.  
One command to run.
```